SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

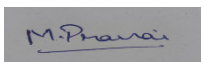
Scenario Based Assignment

Code of Conduct:

I M.Pranai (192525068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search
ii. Analyze the challenges of partial observability and dynamic environments
iii. Describe how the rover builds and updates its internal model
iv. Compare online search with uninformed and informed search strategies
v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem
ii. Analyze the computational challenges of large game trees
iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
iv. Design an evaluation function and justify the factors considered
v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SCENARIO 1: AI-POWERED EMERGENCY MEDICINE DELIVERY

The drone must deliver medicine through a flood-affected region where roads can become blocked and travel costs vary with terrain and weather. The search objective is not merely to find a geographically short route; it is to find a route that is fast, feasible, and acceptably safe using the latest available information.

i. Greedy Best-First Search

Greedy Best-First Search (GBFS) expands the frontier node that appears closest to the destination according to a heuristic. With straight-line distance h(n), it chooses the node with the smallest h(n), ignoring the cost already spent to reach that node. A drone may therefore head towards a destination-facing road even when that road is flooded or crosses difficult terrain.

$$\mathbf{f(n) = h(n)}$$

Strengths: it is often quick, easy to implement, and can reach a goal rapidly when the heuristic points in a useful direction. Weaknesses: it is not generally optimal, can follow an expensive detour, and can be misled by blocked routes or weather changes. Its speed is valuable in emergencies, but speed alone does not guarantee a safe or low-delay delivery.

ii. A* Search

A* combines the actual path cost so far with an estimate of the remaining cost. Let g(n) be the accumulated travel cost and h(n) the estimated remaining cost. The algorithm expands the frontier node with the lowest estimated total cost.

$$\mathbf{f(n) = g(n) + h(n)}$$

For this scenario, g(n) can represent elapsed time or a defined cost that combines time, fuel/energy, and risk. A* therefore avoids preferring a route solely because it looks geographically close. With non-negative edge costs and an admissible heuristic (one that never overestimates the true remaining cost), graph-search A* with appropriate duplicate handling can find an optimal route under its model. If consistency is not guaranteed, the implementation must permit reopening nodes when a better path is found.

iii. Impact of heuristic accuracy

- A useful heuristic directs search towards the destination and can reduce the number of expanded nodes.
- For GBFS, the heuristic largely determines the search order because accumulated cost is ignored. An inaccurate straight-line estimate can make a short-looking but costly or blocked route appear attractive.
- For A*, an admissible and consistent heuristic supports optimality and efficient search. Straight-line distance is admissible for distance only when edge costs are at least the corresponding straight-line distance; if costs are travel time, a suitable lower bound such as distance divided by a valid maximum speed is more appropriate.

iv. Effect of changing blocked paths

A route can become invalid after the initial plan. The drone should incorporate incoming observations, mark confirmed blocked edges as unavailable, update edge costs for weather and terrain, and replan. Re-running A* is straightforward but may repeat work; incremental methods such as D* Lite or Lifelong Planning A* can reuse parts of the previous search when the map changes. Because updates are only partial, the drone should also track the age and confidence of information and use safety margins around uncertain hazards.

v. Recommendation

Use A* as the baseline route planner with a travel-time lower-bound heuristic, a cost model that penalises risk, and frequent replanning as new data arrives. Where blocked paths change often, an incremental replanner such as D* Lite is a stronger operational choice because it can repair the previous plan rather than solve the entire problem from scratch. GBFS may be useful for a quick provisional route, but it should not be the sole safety-critical planner. The system should reject unsafe edges, check route feasibility, and preserve a safe fallback or hover/return policy if communication or map confidence becomes inadequate.

Aspect	**Greedy Best-First Search**	**A* Search**
Priority	h(n) only	g(n) + h(n)
Use of path cost	Ignored in node ranking	Explicitly included
Typical speed	Often fast, but may wander	Can expand more nodes to account for cost
Optimality	Not guaranteed	Guaranteed under suitable cost and heuristic conditions
Dynamic map	Must update/restart or replan	Can replan; incremental variants reuse search work
Emergency use	Fast provisional option	Preferred when cost and safety must be modelled

SCENARIO 2: SMART-CITY TRAFFIC SIGNAL OPTIMISATION

The city must choose signal phases and green-time durations across hundreds of intersections. The state space is large because each intersection has several possible timing plans and traffic conditions interact across the road network. The task is a dynamic, multi-objective optimisation problem.

Formulation and why traditional search is inefficient

Let x encode signal-cycle lengths, phase splits, offsets, and phase sequences for all intersections. Traffic simulation or live sensors estimate delay, queue length, fuel use, and congestion for a candidate plan. A weighted objective can be written as:

$$\textbf{Minimise } J(x) = w_1 D(x) + w_2 F(x) + w_3 C(x)$$

Here D(x) is total vehicle waiting delay, F(x) is estimated fuel consumption, C(x) is a congestion measure, and w_1, w_2, w_3 are non-negative policy weights. The measures should be normalised or expressed in comparable units before weighting. The optimisation is subject to hard constraints such as minimum/maximum green time, safe intergreen/all-red intervals, pedestrian crossing time, and conflict-free signal phases. Emergency-vehicle priority and fairness can be added as constraints or explicitly weighted objectives.

Exhaustive search is inefficient because the number of joint plans grows combinatorially with the number of intersections and candidate settings. Traffic is also time-varying, so a plan evaluated on an old traffic state may become poor minutes later. A practical system uses heuristic optimisation, simulation, and rolling-horizon updates rather than enumerating every possible plan.

i–ii. Hill Climbing and its limitations

Hill Climbing starts from a candidate timing plan, evaluates neighbouring plans (for example, increasing one approach's green time by five seconds), and moves to a neighbour with a better objective value. It repeats until no tested neighbour improves the score. This is simple and can quickly improve a baseline plan, but it is a local search method and normally does not backtrack.

Search issue	Meaning	Traffic-network example
Local optimum / local minimum	No nearby change improves the objective, although a better plan exists farther away.	A small timing change helps one junction, but a coordinated corridor retiming is needed to reduce overall delay.
Plateau	Many neighbouring plans have equal or almost equal scores.	Several green-time settings perform similarly during low traffic, giving the algorithm no clear direction.
Ridge	Improvement requires a sequence of coordinated changes not aligned with a single simple move.	Changing an offset and a downstream green split together improves flow, while either change alone appears worse.

A local maximum is the corresponding concept when the objective is maximised; because this problem is written as minimisation, the more precise term is local minimum. Random restarts, larger neighbourhoods, or coordinated moves can reduce these problems but do not guarantee a global optimum.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA) sometimes accepts a worse neighbouring plan so that the search can escape a local minimum. If the change in objective is $\Delta J = J(\text{new}) - J(\text{current})$, an improving move is accepted; a worsening move may be accepted with probability:

$$\mathbf{P(accept) = \exp(-\Delta J / T),\ \ for\ \Delta J > 0}$$

The temperature T is gradually reduced. At high temperature the search explores more broadly; at low temperature it becomes increasingly selective. SA is relatively memory-efficient and can work with a single candidate, but the cooling schedule and neighbourhood design strongly affect performance.

A Genetic Algorithm (GA) maintains a population of candidate timing plans. It evaluates fitness, selects promising candidates, combines their components through crossover, and introduces variation through mutation. A population can explore different regions of the search space simultaneously and encode coordinated changes across multiple intersections. However, GAs require careful representation, constraint repair, population and mutation settings, and sufficient evaluation time.

iv. Recommendation for scalability and adaptability

A practical recommendation is a hybrid rolling-horizon system: use a Genetic Algorithm or another population-based optimiser offline or in a background process to generate coordinated candidate plans; evaluate them using a calibrated traffic simulator; then apply only a safe near-term portion and re-optimise when sensor data changes. Hill Climbing can be used for rapid local adjustments, while SA is a useful lighter-weight alternative when memory and implementation simplicity are priorities. The final choice should be validated against real-time compute limits and operational safety constraints rather than assuming one metaheuristic is always best.

Method	Strength	Limitation	Suitable role
Hill Climbing	Fast, simple, low overhead	Can stop at a local optimum or plateau	Quick local correction
Simulated Annealing	Can escape local minima by accepting some worse moves	Cooling schedule affects runtime and quality	Single-solution search with controlled exploration
Genetic Algorithm	Population explores multiple regions; supports coordinated plans	More computation and parameter tuning; feasibility must be enforced	Large-scale multi-intersection optimisation
Hybrid rolling horizon	Balances broad search with rapid local updates	Needs careful integration and monitoring	Recommended operational architecture

Practical example: if a stadium event causes a sudden surge on one corridor, the controller can raise the measured queue penalty, generate new candidate offsets for connected intersections, enforce minimum green and pedestrian constraints, simulate the candidates, and safely deploy the best feasible plan. As the surge subsides, the next optimisation cycle returns timings towards normal demand.

SCENARIO 3: ONLINE SEARCH FOR A MARS EXPLORATION ROVER

i. Why an online search agent is required

Offline search assumes the relevant state space and action outcomes are known before planning. A Mars rover has an incomplete map, limited sensing, uncertain terrain, and communication delays. It may discover a crater, loose soil, or a slope only after moving closer. An online search agent alternates between acting, observing, updating its model, and planning again. This allows it to make useful progress without requiring a complete map in advance.

ii. Partial observability and changing conditions

- Partial observability: cameras and other sensors provide incomplete or noisy views; a rock may hide a hazard and distance estimates may be uncertain.
- Uncertain action outcomes: wheel slip, soft soil, or slope can make actual movement differ from the predicted movement.
- Dynamic conditions: lighting, dust, temperature, hardware status, or newly observed hazards can change the safe set of actions.
- Communication limits: the rover cannot rely on continuous human intervention, so it needs onboard autonomy, conservative decisions, and safe-stop behaviour.

The terrain may be physically static, but the rover's knowledge and the operational conditions change over time. It should distinguish a known safe path from an unobserved area rather than treating unknown terrain as certainly traversable.

iii. Building and updating the internal model

The rover maintains an internal map or local cost map containing observed free space, obstacles, terrain traversability, uncertainty, and possible sample sites. Sensor readings are localised and fused with earlier observations. The rover updates cell or edge costs, records confidence, and invalidates routes that intersect newly detected hazards. It then replans from its estimated current state. If localisation confidence falls below a threshold or no safe route exists, it should stop, retreat to a verified safe state, or request assistance.

Online rover search workflow (editable Word table)

Start from the current pose and an incomplete map
↓
Sense local terrain and estimate position / uncertainty
↓
Update map: free space, hazards, terrain cost, confidence
↓
Generate feasible actions and plan to the next waypoint
↓
Safety check: slope, obstacle clearance, energy and risk limits
↓
Execute a short action and monitor actual motion
↓
Compare predicted and observed outcome; update the model
↓
Goal reached? If no, repeat sensing and planning; if unsafe, stop or retreat

iv. Comparison with uninformed and informed search

Approach	Knowledge used	Strength	Limitation in Mars exploration
Uninformed search (e.g., BFS)	Available transitions, without a goal-directed heuristic	Systematic; BFS is complete on finite unweighted graphs	Can expand many states and needs a known graph; cost-insensitive BFS is unsuitable for variable terrain costs
Informed search (e.g., A*)	Path cost plus heuristic estimate to goal	Can guide search efficiently when a map and useful heuristic exist	A fixed map can become stale as new hazards are discovered
Online search	Interleaves action and observation; model grows from experience	Works with incomplete knowledge and can adapt to discoveries	Must manage uncertainty, action risk, and repeated planning
Online informed replanning	Online updates combined with heuristic or incremental planning	Goal-directed and responsive to new local information	Depends on sensor quality, model updates, and safe execution checks

v. Recommendation

Use an online agent with receding-horizon local planning, heuristic goal guidance, uncertainty-aware terrain costs, and a safety supervisor. A* can plan over the currently mapped region; incremental planners can repair routes after map changes. The rover should execute only a limited next segment before sensing again, maintain a verified safe fallback, and treat unknown or low-confidence terrain conservatively. This approach balances progress with adaptability and safety better than a one-time offline route.

SCENARIO 4: UNIVERSITY EXAMINATION TIMETABLING USING CSP

A Constraint Satisfaction Problem (CSP) consists of variables, domains of possible values, and constraints that specify which combinations are permitted. The timetable must satisfy non-negotiable rules first, then optimise preferences such as room utilisation or student convenience.

CSP formulation: variables, domains and constraints

CSP component	Definition for exam scheduling
Variables	For each exam/course e, define time(e), room(e), and where needed invigilator(e). Alternatively, use one compound variable X_e = (time, room) for each exam.
Domains	Allowed exam slots and eligible rooms for each course. Remove slots that conflict with faculty availability, room capacity, accessibility needs, or required precedence.
Hard constraints	No student has overlapping exams; room capacity and room availability are respected; each room hosts at most one exam per slot; required subject ordering is satisfied; faculty and invigilator availability is respected; accommodation requirements are met.
Soft constraints	Avoid too many exams for a student on one day, reduce long gaps, distribute difficult exams, minimise unused room capacity, and honour preferred times where possible.
Objective	Find a complete assignment satisfying all hard constraints, then minimise a penalty score for soft-constraint violations.

Let E be the set of exams, T the set of time slots, and R the set of rooms. A simple compound variable X_e has domain $D_e \subseteq T \times R$. If exams e and f share a student, then:

$$\textbf{studentConflict(e,f)} \Rightarrow \textbf{time(e)} \neq \textbf{time(f)}$$

For room capacity and exclusivity:

$$\textbf{students(e)} \leq \textbf{capacity(room(e))}$$

$$\textbf{time(e) = time(f)} \wedge \textbf{room(e) = room(f)} \Rightarrow \textbf{e = f}$$

For a precedence requirement e before f, require start(time(e)) < start(time(f)). A soft-constraint objective can be expressed as minimising the sum of weighted penalties, $J = \Sigma_i \, w_i \, p_i(X)$, subject to all hard constraints. Hard constraints must never be traded away merely to improve the soft score.

ii. Backtracking search

Backtracking builds an assignment incrementally. It selects an unassigned exam, assigns a value from its current domain, and checks consistency with the partial timetable. If a conflict occurs or a later variable has no legal value, it undoes the most recent assignment and tries another value. Variable ordering such as Minimum Remaining Values (MRV) selects the exam with the fewest legal options; degree heuristic breaks ties by choosing the exam that constrains the most other exams. Least Constraining Value (LCV) tries the value that removes the fewest options from neighbours first.

iii. Constraint propagation and forward checking

Forward checking removes values from the domains of unassigned neighbouring exams whenever an assignment is made. For example, placing a course in Room A at 9:00 removes 9:00/Room A from the domains of other exams that conflict through room exclusivity or student overlap. If any unassigned exam's domain becomes empty, the algorithm detects failure immediately and backtracks instead of exploring deeper.

- Constraint propagation reduces the remaining search space before the next assignment.
- Arc consistency can go further than forward checking by ensuring that each remaining value has compatible support in neighbouring variables.
- The system must restore removed domain values correctly during backtracking, commonly using a trail or copy of domain changes.

CSP timetable workflow (editable Word table)

Collect courses, enrolments, rooms, faculty availability and accommodations
↓
Create variables, domains, hard constraints and soft penalties
↓
Pre-process domains; apply constraint propagation
↓
Select an unassigned exam (MRV / degree heuristic)
↓
Try a value (least-constraining feasible slot and room)
↓
Propagate constraints and test for an empty domain
↓
Consistent? Yes → recurse; No → undo and try next value
↓
All exams assigned? Yes → validate hard constraints and improve soft score
↓
If no value remains, backtrack; if search limit reached, report status honestly

iv. Scalability and real-world challenges

Timetabling grows difficult because courses share many students, rooms differ in capacity and equipment, staff availability is limited, and accommodation needs can sharply restrict domains. Late enrolment changes or room closures may invalidate an otherwise feasible schedule. A scalable solution should use conflict graphs, strong variable ordering, constraint propagation, decomposition by campus or exam period where dependencies allow, and local repair when changes occur. Large instances may combine CSP methods with local search or integer programming, but every final schedule must be independently checked against all hard constraints.

Challenge	**Practical response**
Large number of courses and conflicts	MRV, degree heuristic, conflict graph and propagation
Restricted rooms / accessibility	Filter domains before search; encode accommodation as a hard constraint
Student clashes	Build conflict edges from enrolment data and validate all assigned slots
Late changes	Use incremental repair and re-run validation
Soft preferences	Optimise a weighted penalty only after preserving hard feasibility

Recommended strategy: use backtracking CSP with MRV, degree heuristic, LCV, forward checking, and stronger propagation where its cost is justified. For very large instances, decompose carefully and apply local improvement to a feasible timetable. The system should report infeasibility or a best-found feasible schedule rather than silently violating a hard constraint.

SCENARIO 5: STRATEGIC GAME AI USING MINIMAX AND ALPHA-BETA PRUNING

i. Minimax model

Minimax models a two-player, turn-taking, zero-sum game in which one player (MAX) tries to maximise utility and the opponent (MIN) tries to minimise it. The game is represented as a tree: nodes are game states and edges are legal actions. Terminal states use their true utility; non-terminal states use recursive minimax values.

$$\mathbf{V(s) = U(s), \text{ if } s \text{ is terminal}}$$

$$\mathbf{V(s) = \max_a V(Result(s,a)), \text{ for MAX}}$$

$$\mathbf{V(s) = \min_a V(Result(s,a)), \text{ for MIN}}$$

The algorithm assumes both players choose optimally within the searched tree. It is appropriate for adversarial games but does not by itself model uncertainty, simultaneous actions, or opponents who do not follow the assumed policy.

ii. Computational challenges

If the average branching factor is b and the search depth is d, a full game tree has $O(b^d)$ nodes in the worst case. Even moderate values become expensive for real-time strategy games with many units and possible actions. Additional challenges include time limits, large state representations, repeated equivalent states, and the fact that a cutoff state is not terminal and therefore needs an approximate evaluation. A deeper search can be more accurate but must finish before the move deadline.

iii. Alpha-Beta pruning

Alpha-Beta pruning returns the same minimax decision as full minimax when node values are exact and the same search depth is used, but it avoids exploring branches that cannot change the result. Alpha (α) is the best value MAX can guarantee on the current path; beta (β) is the best value MIN can guarantee. At a MAX node, update $\alpha = \max(\alpha, \text{child value})$. At a MIN node, update $\beta = \min(\beta, \text{child value})$. When $\alpha \geq \beta$, remaining children can be pruned.

$$\mathbf{\text{Prune when } \alpha \geq \beta}$$

With poor move ordering, Alpha-Beta's worst-case complexity remains $O(b^d)$. With ideal ordering, the effective number of explored nodes can approach O(b^(d/2)), allowing deeper search in the same time. Good move ordering is therefore important.

Worked pruning example

Consider a root MAX node A with two MIN children B and C. B has terminal leaves 3 and 5, so $B = \min(3, 5) = 3$. After B is evaluated, root A has $\alpha = 3$. Next evaluate C, whose first child returns 2, so at C (a MIN node) $\beta = 2$. Since $\beta \leq \alpha$ ($2 \leq 3$), the remaining child of C cannot make C exceed 2; MAX already has a guaranteed option worth 3 through B. Therefore the other child of C is pruned.

Worked tree (terminal scores; editable representation)

A: MAX → current α = 3 after evaluating B

↓

B: MIN → leaves 3 and 5 → B = 3

↓

C: MIN → first leaf = 2 → β = 2

↓

At C: β ≤ α (2 ≤ 3) → prune C's remaining child

↓

Root chooses B; value at A = max(3, value(C)) = 3

Node	Type	Observed children	Current bound / value	Action
A	MAX	B evaluated as 3	α = 3	Continue to C
B	MIN	3, 5	β becomes 3; B = 3	Return 3 to A
C	MIN	First leaf = 2	β = 2; α at A = 3	Prune remaining child because β ≤ α
A	MAX	B = 3; C ≤ 2	A = 3	Choose B

iv. Evaluation function

When the search reaches a depth limit before a terminal state, an evaluation function estimates how favourable the position is to MAX. A practical linear function for a strategy game is:

$$\mathbf{Eval(s) = w_1M + w_2R + w_3T + w_4P + w_5O - w_6C}$$

where M = relative military strength (effective units and health), R = resource advantage, T = territory or objective control, P = positional advantage (cover, range, mobility and formation), O = objective progress or strategic points, and C = expected cost/risk from exposed units or imminent threats. Each feature is computed as MAX's value minus MIN's value where appropriate. Weights w_i reflect game design and should be tuned and tested; they are not universal constants. Terminal win/loss/draw scores should dominate non-terminal heuristic scores, and the function should avoid double-counting correlated features.

Feature	Why it matters	Example signal
Military strength (M)	Captures remaining combat capability	Healthy units and advantageous unit trades
Resources (R)	Represents ability to build and sustain forces	Resource stock minus opponent stock

Territory / objectives (T, O)	Rewards control of strategically valuable areas	Owned objectives, score progress, supply access
Position (P)	Captures tactical quality beyond unit count	Cover, line of sight, mobility, choke-point control
Risk / cost (C)	Penalises vulnerable units and costly actions	Threatened units, expected losses, supply exposure

v. Recommended real-time strategy

Use iterative-deepening Minimax with Alpha-Beta pruning, strong move ordering, and a fixed time budget. The system first obtains a valid shallow answer, then searches deeper while time remains; if the deadline approaches, it returns the best move from the last completed iteration. A transposition table can cache previously evaluated states, while move ordering can test promising tactical actions first. At depth limits, use the evaluation function above and, where suitable, a limited quiescence search to avoid evaluating unstable tactical positions prematurely. If the game has simultaneous actions or stochastic outcomes, the model must be extended rather than applying standard Minimax without qualification.

Method	**Decision quality**	**Computational cost**	**Role**
Minimax	Optimal within the fully searched tree under assumptions	$O(b^d)$ worst-case	Baseline adversarial search
Alpha-Beta	Same minimax result at the same depth	Worst $O(b^d)$; best-ordering near O(b^(d/2))	Recommended pruning method
Iterative deepening + Alpha-Beta	Returns a usable answer early and improves it with time	Repeated searches, often offset by ordering/cache benefits	Real-time deadline management

CONCLUSION

These five scenarios illustrate how AI search and optimisation methods must be selected according to the structure and constraints of the problem. A* with incremental replanning is suitable for medicine delivery because path cost and changing hazards matter. Traffic-signal optimisation benefits from metaheuristic exploration combined with a rolling-horizon controller and hard safety constraints. A Mars rover requires online, uncertainty-aware planning that repeatedly senses and updates its map. University timetabling is naturally modelled as a CSP, where backtracking and constraint propagation enforce feasibility before soft preferences are optimised. For adversarial game decisions, Alpha-Beta pruning preserves Minimax decisions at the searched depth while reducing unnecessary work, especially when combined with iterative deepening and good move ordering. Across all cases, reliable performance depends on a clear cost or evaluation model, current information, explicit safety/feasibility checks, and honest handling of computational limits.

REFERENCES

Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Koenig, S., & Likhachev, M. (2002). D* Lite. Proceedings of the AAAI Conference on Artificial Intelligence, 476–483.

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Holland, J. H. (1992). Adaptation in Natural and Artificial Systems. MIT Press. (Original work published 1975.)

Mackworth, A. K. (1977). Consistency in networks of relations. Artificial Intelligence, 8(1), 99–118. https://doi.org/10.1016/0004-3702(77)90007-8

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial Intelligence, 6(4), 293–326. https://doi.org/10.1016/0004-3702(75)90019-3